ComicCraft – Requirement Analysis

1. Project Overview

ComicCraft is a Generative AI-based application that transforms user-provided stories into structured comic panels and combines the generated panels into a final comic. The project is developed as part of the Naan Mudhalvan program to demonstrate the practical application of Artificial Intelligence, Natural Language Processing (NLP), Large Language Models (LLMs), image generation, and full-stack application development.

The main objective is to simplify comic creation by automating story analysis, scene segmentation, panel planning, illustration generation, and comic composition through an integrated workflow.

Project Name: ComicCraft

Program: Naan Mudhalvan

Project Phase: Requirement Analysis

Project Type: Story Input → Comic Panels → Final Comic

Primary Input: User-provided story text

Primary Output: A composed comic containing illustrated panels and optional captions or dialogue

2. Purpose of Requirement Analysis

Requirement analysis identifies what ComicCraft must do, how users will interact with it, which technical resources are required, and what conditions define successful operation.

This phase establishes the scope and expected behavior of the application before detailed system design and implementation. It also identifies technical limitations, dependencies, quality expectations, and possible risks associated with Generative AI services.

3. Problem Statement

Traditional comic creation requires a combination of storytelling, illustration, character design, scene planning, dialogue placement, and page layout skills. Producing a complete comic manually can take considerable time and may be difficult for users without artistic experience.

ComicCraft addresses this problem by accepting a written story and converting it into scenes, panel descriptions, AI-generated illustrations, and a final comic layout. The application aims to reduce manual effort while allowing users to review and revise generated content.

4. Project Objectives

- Accept a story from the user through a simple interface.
- Validate the story input before processing.
- Identify important characters, locations, events, and actions.
- Divide the story into logical scenes.
- Create a structured plan for comic panels.
- Generate visual prompts for individual panels.
- Request illustrations from a compatible image-generation model or API.
- Maintain consistent character descriptions and visual context where supported.
- Arrange panels in the intended reading order.
- Add captions and dialogue using the composition layer when supported.
- Display a preview of the generated comic.
- Allow users to revise the story or regenerate selected panels where implemented.
- Provide a final output in a supported image or document format.

5. Project Scope

5.1 In Scope

The planned scope of ComicCraft includes:

1. Story text submission.
2. Input validation and error reporting.
3. AI-assisted story understanding.
4. Character, location, and event identification.
5. Scene segmentation.
6. Comic panel planning.
7. Visual prompt generation.
8. AI-based illustration generation.
9. Comic panel ordering and composition.
10. Previewing the completed comic.
11. Optional panel regeneration and editing.

12. Saving or exporting the final comic, depending on implementation.

5.2 Out of Scope for the Initial Version

The following features are not mandatory for the initial prototype:

- Professional hand-drawn animation.
- Real-time collaborative comic editing.
- Advanced frame-by-frame animation.
- A complete publishing or marketplace platform.
- Guaranteed identical character appearance across every generated image.
- Training a new foundation model from scratch.
- Unlimited generation without API usage limits.
- Cloud-based project synchronization unless explicitly implemented.

These features may be considered for future development.

6. Stakeholders and Target Users

Stakeholder	Role or Interest
Students	Learn about Generative AI and create visual stories
Writers	Convert written stories into comic-style narratives
Beginner creators	Create comics without advanced drawing skills
Educators	Explore visual storytelling for learning materials
Project developers	Design, implement, test, and maintain the application
Project evaluators	Assess the problem, requirements, implementation, and results
AI service providers	Supply language or image-generation capabilities where integrated

7. User Requirements

The user should be able to:

- Understand how to use the application without specialized technical knowledge.

- Enter or paste a story into the interface.

- Submit valid story text for processing.

- Receive useful feedback if the input is invalid.

- View generated scene or panel descriptions when supported.

- View the illustrations created from the story.

- Preview the final comic in sequence.

- Identify and recover from common generation errors.

- Revise the story or regenerate panels if the application supports editing.

- Save or export the result if the corresponding feature is available.

8. Functional Requirements

Functional requirements describe the operations that ComicCraft is expected to perform.

| ID | Requirement | Priority | Acceptance Condition |

|---|---|---|---|

| FR-01 | Accept story input | High | The user can enter or paste story text |

| FR-02 | Validate input | High | Empty or invalid input produces a clear message |

| FR-03 | Submit story | High | Valid input can be sent for processing |

| FR-04 | Analyze story | High | The processing component extracts a structured narrative representation |

| FR-05 | Identify characters and settings | High | Relevant entities are represented in the scene plan when present in the story |

| FR-06 | Segment scenes | High | The story is divided into ordered scenes |

| FR-07 | Plan comic panels | High | Scene information is converted into panel descriptions |

| FR-08 | Generate prompts | High | Each planned panel has an associated visual prompt |

| FR-09 | Generate illustrations | High | The configured image service returns panel images or a handled error |

| FR-10 | Maintain panel order | High | The composition follows the planned sequence |

| FR-11 | Compose final comic | High | Available panels are combined into a final layout |

| FR-12 | Preview comic | High | The user can view the composed result |

| FR-13 | Handle generation failures | High | Failures are reported without an unhandled application crash |

| FR-14 | Regenerate panels | Medium | The user can regenerate a selected panel if implemented |

| FR-15 | Edit captions or dialogue | Medium | Text can be modified if editing is implemented |

| FR-16 | Export final comic | Medium | The comic can be downloaded in a supported format if implemented |

| FR-17 | Report progress | Medium | The interface communicates processing status where supported |

Priority definitions: High = required for the core workflow; Medium = desirable and may depend on the implemented version.

9. Non-Functional Requirements

9.1 Usability

- The interface should be simple and easy to understand.
- Story submission should require minimal steps.
- Error messages should explain the problem in understandable language.
- The preview should display panels in the correct sequence.

9.2 Performance

- The application should avoid unnecessary language-model and image-generation requests.
- The backend should handle long-running generation requests appropriately.
- Processing progress should be communicated where practical.
- Performance expectations should account for external model latency and network conditions.

9.3 Reliability

- Invalid requests should be handled safely.
- Failed API calls should produce meaningful error messages.
- Network timeouts and rate limits should be considered.
- A failed panel should not silently produce an incorrect final comic.
- Retry behavior should be limited to avoid repeated costs or duplicate requests.

9.4 Security

- API keys must be stored in environment variables or a suitable secret manager.
- The real `.env` file must not be committed to GitHub.
- User input must be validated before processing.
- Errors must not expose secrets or internal stack traces to ordinary users.

- External services must be used according to their terms and privacy policies.

9.5 Maintainability

- Frontend, backend, AI integrations, and composition logic should be separated into manageable modules.
- Dependencies should be documented.
- Configuration should be centralized where practical.
- Code should be readable and organized consistently.

9.6 Scalability

- The architecture should allow the LLM or image provider to be replaced with limited code changes.
- Panel counts and story lengths should be configurable within safe limits.
- Resource-intensive tasks should be managed according to the capabilities of the deployment environment.

9.7 Portability

- The project should support a documented local development environment.
- Installation instructions should identify the required Python version, dependencies, environment variables, and startup command.
- Deployment-specific settings should be separated from application logic.

9.8 Accessibility

- Interface text should be readable.
- Input fields and buttons should have clear labels.
- Images should have appropriate alternative text where supported.
- The application should remain usable on common screen sizes where responsive design is implemented.

10. System Requirements

10.1 Hardware Requirements

| Component | Suggested Requirement |

|---|---|

| Processor | Modern dual-core processor or better |

| Memory | 4 GB RAM minimum for basic development; 8 GB or more recommended |

| Storage | Enough free space for dependencies, generated images, and project files |

| Network | Internet access when using hosted AI APIs |

| Graphics processor | Optional for hosted models; local model requirements vary |

These are suggested development requirements, not benchmarked minimums. Local AI models may require substantially more memory and graphics resources.

10.2 Software Requirements

| Software | Purpose |

|---|---|

| Windows, Linux, or macOS | Development operating system |

| Visual Studio Code | Code editing and project development |

| Python | Backend and AI integration, if using a Python backend |

| pip and virtual environment | Dependency installation and isolation |

| Git | Version control |

| GitHub | Source code hosting and documentation |

| Web browser | Accessing and testing the application |

| LLM API or compatible model | Story analysis and panel planning |

| Image-generation API or compatible model | Panel illustration generation |

The exact versions and providers must be confirmed against the final project configuration.

11. Input and Output Specifications

11.1 Input Specification

| Field | Description | Validation |

|---|---|---|

| Story text | Main narrative supplied by the user | Required; must not be empty or whitespace only |

| Art style | Optional visual style preference | Must be supported by the selected generation workflow |

| Panel count | Optional desired number of panels | Must be within configured limits |

| Language | Optional story or dialogue language | Must be supported by the processing workflow |

| Character references | Optional descriptions or reference images | Must comply with supported formats and service rules |

The initial implementation may support only story text. Optional fields should be included only if implemented.

11.2 Output Specification

| Output | Description |

|---|---|

| Story analysis | Structured summary, characters, locations, and events where supported |

| Scene plan | Ordered list of scenes derived from the story |

| Panel plan | Descriptions and prompts for individual comic panels |

| Panel illustrations | Generated images for planned panels |

| Final comic | Ordered and composed comic page or pages |

| Error response | Clear information about validation or generation failures |

11.3 Data Flow

1. Story text enters through the frontend.

2. The backend validates and processes the request.

3. The language model produces structured scene and panel information.

4. The prompt builder prepares descriptions for illustration generation.

5. The image service returns illustrations.

6. The composition engine arranges the illustrations and supported text elements.

7. The frontend displays the completed comic.

12. Use Cases

Use Case 1: Generate a Comic

Actor: User

Precondition: The application is accessible and required AI services are configured.

Main flow:

1. The user opens ComicCraft.
2. The user enters a story.
3. The application validates the input.
4. The user submits the story.
5. The application analyzes the narrative.
6. The application plans scenes and panels.
7. Illustrations are generated.
8. The panels are arranged into a comic.
9. The user views the final result.

Alternative flow: If the story is invalid, the application asks the user to correct it. If a generation service fails, the application reports the problem and offers a supported retry or recovery option.

Postcondition: A final comic is available for preview if the workflow succeeds.

Use Case 2: Regenerate a Panel

Actor: User

Precondition: A comic panel has been generated and panel regeneration is implemented.

Main flow:

1. The user selects a panel.
2. The user requests regeneration or modifies the prompt.
3. The application submits the updated request.
4. A replacement image is generated.
5. The composition is updated.

Postcondition: The updated panel appears in the comic preview.

Use Case 3: Export the Comic

Actor: User

Precondition: A comic has been composed and export is implemented.

Main flow:

1. The user selects an export option.
2. The application prepares the comic in a supported format.
3. The application provides the file to the user.

Postcondition: The user receives the generated comic file.

13. User Stories

ID	User Story	Expected Benefit
US-01	As a user, I want to enter a story so that I can convert it into a comic.	Easy story submission
US-02	As a user, I want the story divided into scenes so that the narrative is easier to visualize.	Structured storytelling
US-03	As a user, I want each scene represented by comic panels so that the story becomes visual.	Clear panel planning
US-04	As a user, I want AI-generated illustrations so that I do not have to draw every panel manually.	Reduced manual effort
US-05	As a user, I want the panels arranged in order so that the comic follows the story.	Narrative continuity
US-06	As a user, I want to preview the comic so that I can check its quality.	Quality review
US-07	As a user, I want understandable error messages so that I can recover from failures.	Better usability
US-08	As a user, I want to export the comic if supported so that I can keep or share the result.	Reusable output

14. Mermaid Workflow Diagram

The diagram below represents the expected functional workflow from story input to the final comic.

```mermaid
flowchart TD
    A([Start]) --> B[Open ComicCraft]
    B --> C[Enter Story]
    C --> D[Validate Input]
    D --> E{Valid Story?}
```

```
    E -- No --> F[Display Validation Error]
    F --> C
    E -- Yes --> G[Submit Story to Backend]
    G --> H[Analyze Story with LLM]
    H --> I[Identify Characters and Settings]
    I --> J[Segment Story into Scenes]
    J --> K[Create Panel Plan]
    K --> L[Generate Visual Prompts]
    L --> M[Generate Panel Images]
    M --> N{Generation Successful?}
    N -- No --> O[Report Error or Retry]
    O --> M
    N -- Yes --> P[Arrange Panels]
    P --> Q[Add Captions and Dialogue if Supported]
    Q --> R[Compose Final Comic]
    R --> S[Preview Comic]
    S --> T{Satisfied?}
    T -- No --> U[Revise Story or Panels]
    U --> H
    T -- Yes --> V[Export if Supported]
    V --> W([End])
```

15. High-Level System Architecture

The application can be divided into the following components:

- **Frontend:** Collects story input and displays generated content.
- **Backend API:** Validates requests, coordinates processing, and returns responses.
- **Story Processing Module:** Converts the narrative into structured scenes and panel descriptions.
- **LLM Integration:** Provides story analysis and text generation.
- **Prompt Builder:** Creates image-generation prompts from the panel plan.
- **Image Generation Module:** Requests illustrations from a compatible image model or service.
- **Composition Module:** Arranges panels and adds supported text elements.

- **Output Module:** Displays or exports the completed comic.

```mermaid
flowchart LR
    USER[User] --> FRONT[Frontend]
    FRONT --> API[Backend API]
    API --> VALID[Input Validation]
    VALID --> PROCESS[Story Processing]
    PROCESS --> LLM[LLM Service]
    PROCESS --> PLAN[Scene and Panel Planner]
    PLAN --> PROMPT[Prompt Builder]
    PROMPT --> IMAGE[Image Generation Service]
    IMAGE --> ASSETS[Generated Image Assets]
    ASSETS --> COMPOSE[Comic Composition]
    COMPOSE --> OUTPUT[Final Comic]
    OUTPUT --> FRONT
```

16. Data Requirements

The application may need to manage the following information during processing:

- Original story text.
- Story summary or structured narrative data.
- Character descriptions and locations.
- Ordered scene descriptions.
- Panel descriptions and image prompts.
- Generated image files or references to those files.
- Panel order and layout settings.
- Captions and dialogue, where supported.
- Generation status and error information.

Persistent storage is not mandatory for the initial prototype. If storage is implemented, the project should define file naming, retention, access control, and deletion behavior. Sensitive user content should not be retained longer than necessary.

17. API and Integration Requirements

The backend should expose endpoints appropriate to the chosen implementation. For a FastAPI-based version, possible endpoint responsibilities include:

Endpoint Example	Method	Purpose
`/`	GET	Display the main interface
`/generate`	POST	Accept story input and start comic generation
`/health`	GET	Report application health, if implemented
`/regenerate-panel`	POST	Regenerate a selected panel, if implemented
`/download/{comic_id}`	GET	Provide an exported comic, if implemented

These are proposed endpoint examples, not confirmed routes. Actual endpoint paths, methods, request schemas, and responses must match the source code.

Integration requirements include valid credentials, supported model configuration, network access for hosted services, response validation, timeouts, error handling, and compliance with provider limits.

18. Assumptions and Dependencies

Assumptions

- Users provide text that can be processed as a story.
- A compatible language model is available for story analysis.
- A compatible image-generation model or service is available for illustration generation.
- The selected models can process the requested language and content within their limitations.
- The user has access to a supported web browser and the running application.

Dependencies

- Programming language and backend framework.
- Frontend libraries or server-rendered templates.
- Language-model provider or local model.
- Image-generation provider or local model.
- Image-processing libraries.
- Network connectivity when external services are used.
- API credentials, usage quotas, and service availability where applicable.

19. Constraints and Limitations

- Generated illustrations may differ from the written story.
- Character appearance may vary between panels.
- Text rendered directly inside generated images may be inaccurate.
- Long stories may need input limits or segmentation.
- Image generation can take time and incur costs.
- Hosted services may impose rate limits or usage restrictions.
- Export options depend on the implemented composition and file-processing modules.
- The final comic quality depends on the prompts, model capabilities, and user review.

20. Risk Analysis

Risk	Impact	Mitigation
AI service unavailable	Comic generation may fail	Display errors and support controlled retries
Invalid API credentials	Requests may be rejected	Validate configuration and document setup steps
Inconsistent characters	Reduced visual continuity	Reuse descriptions and references where supported
Excessive generation time	Poor user experience	Provide progress feedback and configurable limits
High API costs	Increased project expense	Limit panel counts and monitor usage
Malformed model output	Panel planning may fail	Validate structured responses before composition
Secret leakage	Unauthorized API usage	Exclude credentials from source control
Image-processing failure	Final comic may not be created	Validate image files and handle composition errors
Unintended content	Unsuitable output	Apply suitable input and output safeguards

21. Acceptance Criteria

The core application can be considered ready for initial demonstration when the following criteria are satisfied:

- [ ] The application starts using the documented setup procedure.
- [ ] The main interface accepts story text.
- [ ] Empty input is rejected with a clear message.
- [ ] Valid story input reaches the backend.
- [ ] The story-processing module produces a usable scene or panel plan.
- [ ] The prompt-generation component creates panel prompts.
- [ ] The configured image service generates panel illustrations or returns a handled error.
- [ ] The application arranges successfully generated panels in the intended order.
- [ ] The final comic can be previewed.
- [ ] Common API and image-processing errors are handled.
- [ ] Credentials are stored outside committed source files.
- [ ] The documentation accurately describes the implemented features.

Optional editing, regeneration, persistent storage, and export criteria should be added when those features are implemented.

22. Testing Requirements

Testing should cover the following areas:

1. **Input Testing:** Empty stories, whitespace, short stories, long stories, and unsupported inputs.
2. **Story Analysis Testing:** Correct extraction of major events, characters, and locations.
3. **Scene Planning Testing:** Logical scene boundaries and preservation of narrative order.
4. **Prompt Testing:** Clear descriptions that reflect the intended scene.
5. **Image Service Testing:** Successful generation, timeouts, invalid credentials, rate limits, and service failures.
6. **Composition Testing:** Correct panel order, image dimensions, spacing, and readable layout.
7. **API Testing:** Valid requests, invalid requests, status codes, and error responses.
8. **End-to-End Testing:** A complete story submission through final comic preview.
9. **Security Testing:** Confirmation that API credentials are not exposed in GitHub.
10. **Usability Testing:** Confirm that a beginner can understand the interface and error messages.

23. Development Priorities

Phase 1 – Core Workflow

- Create the story input interface.
- Implement backend input validation.
- Integrate story analysis and panel planning.
- Integrate image generation.
- Compose and preview the final comic.

Phase 2 – Reliability and Quality

- Improve prompt quality and structured output validation.
- Add clear progress and error messages.
- Improve character consistency.
- Test larger stories and multiple panels.

Phase 3 – Optional Enhancements

- Panel editing and regeneration.
- Multiple art styles.
- Captions and speech bubbles.
- PNG or PDF export.
- Project history and saved comics.
- Regional language support.

24. Detailed Summary

ComicCraft is proposed as a Generative AI application that converts ordinary written stories into illustrated comic panels and a final comic composition. Requirement analysis establishes the expected behavior of the system, identifies its users, defines its functional and non-functional requirements, and specifies how the application should be tested.

The primary workflow begins when a user enters a story through the frontend. The backend validates the text and passes it to a story-processing component. A language model analyzes the narrative and helps identify characters,

settings, events, and scenes. The application then creates an ordered panel plan and prepares prompts for illustration generation. A compatible image-generation service produces the panel images, which are combined by the composition component into a final comic.

The application should provide understandable validation messages, handle AI-service failures, protect credentials, and allow users to inspect the result. Editing, regeneration, persistent storage, and export are useful optional capabilities whose availability depends on the implementation.

The main quality goals are usability, reliability, security, maintainability, performance, and visual continuity. Testing should verify each component individually and the complete story-to-comic workflow. The main technical risks include model errors, inconsistent illustrations, external service outages, processing time, and API costs. These risks can be reduced through structured prompts, response validation, sensible limits, controlled retries, and clear user feedback.

This document serves as the requirements baseline for the next project phase: system design. The final repository should update all proposed requirements and example endpoints to reflect the actual implementation and verified test results.

25. Conclusion

The requirement analysis defines the expected scope and behavior of ComicCraft as a story-to-comic Generative AI system. It provides a foundation for architecture design, implementation, testing, and documentation under the Naan Mudhalvan project.

By combining story understanding, scene segmentation, panel planning, image generation, and comic composition, ComicCraft aims to make visual storytelling more accessible to students, writers, and beginner creators.

26. Document Information

- **Project:** ComicCraft
- **Program:** Naan Mudhalvan
- **Phase:** Requirement Analysis
- **File:** `2_Requirement_Analysis/README.md`
- **Input:** Story text
- **Process:** Story analysis → Scene planning → Panel prompts → Image generation → Comic composition
- **Output:** Final comic
- **Status:** Requirements documentation; implementation details must be verified against the actual codebase

---

Documentation note: Requirements, endpoint examples, and optional features in this document are planning specifications. Update them as needed to match the implemented application before final submission.